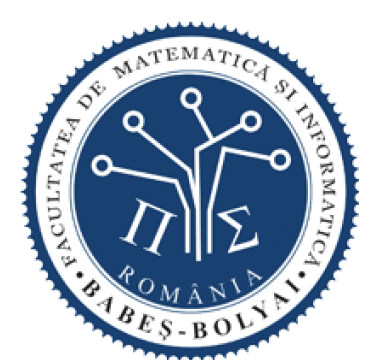




Computer systems architecture

01 - Data representation: Elementary data types, binary representation, data organization and storage.

Mihai Suciu

Faculty of Mathematics and Computer Science

Structure

- Courses 2 h/week
- Seminars 1 h/week
- Lab 2 h/week
- course web page: <https://www.cs.ubbcluj.ro/~mihai-suciu>
- materials will be posted on MsTeams (access code 6piibsx) and on the course web page
- activities will take place according to <https://www.cs.ubbcluj.ro/files/orar/2025-1/disc/MLE5004.html>

Team

- course: Mihai Suciu (mihai.suciu [at] ubbcluj.ro)
- seminar:
 - Lect. Dr. Alina Călin
 - C.d.asociat Nicolae Bodea
- lab:
 - Lect. Dr. Alina Călin
 - C.d.asociat Andrei Ioja Iosif
 - C.d.asociat Andrea Talpos
 - C.d.asociat Daniel Todasca
 - C.d.asociat Flavius Ilinoiu
 - C.d.asociat Nicolae Bodea
 - C.d.asociat Tiberius Mărginean
 - C.d.asociat Toma Furdui
 - C.d.asociat Tudor Lechintan

Evaluation

- lab:
 - 3 tests (~ weeks 5, 9, and 13/14)
 - bonus for extra work
- written exam
- final grade:

$$\text{Grade} = 0.6 * E + 0.4 * L + B$$

where:

E = written exam

L = lab

B = bonus

Minimum standard of performance

$$\text{Grade} = 0.6 * E + 0.4 * L + B$$

- Grade: minimum 5 at each grading activity.
- Attendances: 75% attendance at seminar activities, 90% attendance at laboratory activities.
- Students with more than 2 unmotivated absences at the seminar/laboratory activities will not be able to take the exam in the normal session and or in the retake examination session

Let's begin

- How does a computer work?
- How do you talk (give instructions) to a computer?

```
01001000 01100101 01101100 01101100 01101111 00100000  
01100011 01110011 01100001 00100001
```

Number systems

- A major stumbling block many beginners encounter when attempting to learn assembly language is the common use of the binary and hexadecimal numbering systems
- positional number systems using various radices (bases)
- Most modern computer systems do not represent numeric values using the decimal (base-10) system

Number systems

- Numerical data are expressed in various positional number systems for each radix, or base
- A positional number system encodes a vector of n bits
- The encoded vector is also associated with a radix r , $r \geq 2$

$$A = (a_{n-1} a_{n-2} a_{n-3} \dots a_1 a_0)$$

- where $0 \leq a_i \leq r - 1$

$$A = \sum_{i=0}^{n-1} a_i r^i$$

Number systems

- adding 1 to the highest digit in a radix r gives

$0, 1, 2, \dots, (r-1), 10, 11, 12, \dots, 1(r-1), \dots$

- left table shows the counting sequence for different radices

Radix (base)	Character Sets for Different Radices
2	{0, 1}
3	{0, 1, 2}
4	{0, 1, 2, 3}
5	{0, 1, 2, 3, 4}
6	{0, 1, 2, 3, 4, 5}
7	{0, 1, 2, 3, 4, 5, 6}
8	{0, 1, 2, 3, 4, 5, 6, 7}
9	{0, 1, 2, 3, 4, 5, 6, 7, 8}
10	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
11	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A}
12	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B}
13	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C}
14	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D}
15	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E}
16	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}

Decimal	$r = 2$	$r = 4$	$r = 8$
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	10	4
5	101	11	5
6	110	12	6
7	111	13	7
8	1000	20	10
9	1001	21	11
10	1010	22	12
11	1011	23	13
12	1100	30	14
13	1101	31	15
14	1110	32	16
15	1111	33	17
16	10000	100	20
17	10001	101	21

Number systems

- Example: count from decimal 0 to 25 in radix 5, 12

Binary Number System

- radix is 2 in the binary number system
- The binary number system is the most conventional and easily implemented system for internal use in a digital computer
- Ease of implementation and the speed of execution in binary of the four basic operations: addition, subtraction, multiplication, and division

$$2^{n-1} 2^{n-2} \dots 2^3 2^2 2^1 2^0 . 2^{-1} 2^{-2} 2^{-3} \dots 2^{-m}$$

Binary Number System

- example $1011.101_2 \rightarrow$ base 10
 - $r = 2$ and $a_i \in \{0,1\}$ for $-m \leq i \leq n-1$

$$\begin{array}{ccccccc} 2^3 & 2^2 & 2^1 & 2^0 & . & 2^{-1} & 2^{-2} & 2^{-3} \\ 1 & 0 & 1 & 1 & . & 1 & 0 & 1 \end{array}_2 = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + \\ (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) \\ = 11.625_{10}$$

Binary Number System

- Digital systems are designed using bistable storage devices that are either reset (logic 0) or set (logic 1).
- These bistable devices can be concatenated to any length n to store binary data.
 - 1 byte (8 bits) of data, eight bistable storage devices are required , e.g.
0110 1011 (107_{10})

Binary Number System

Decimal	Binary			
	8	4	2	1
	2^3	2^2	2^1	2^0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

101 00000101 0000000000101 ...0000000000000101

The Hexadecimal Numbering System

- Binary numbers are verbose
- Unfortunately, the computer “thinks” in binary, so most of the time it is convenient to use the binary numbering system.
- Hexadecimal numbers offer the two features we’re looking for:
 - They’re very compact
 - It’s simple to convert them to binary and vice versa

$$\begin{array}{cccccccc} 16^3 & 16^2 & 16^1 & 16^0 & . & 16^{-1} & 16^{-2} & 16^{-3} & 16^{-4} \\ 6 & A & 8 & C & . & D & 4 & 1 & 6 \end{array} = \begin{array}{l} (6 \times 16^3) + (10 \times 16^2) + (8 \times 16^1) \\ + (12 \times 16^0) + (13 \times 16^{-1}) + (4 \times 16^{-2}) \\ + (1 \times 16^{-3}) + (6 \times 16^{-4}) \\ = 27,276.82846_{10} \end{array}$$

The Hexadecimal Numbering System

Decimal	Hexadecimal	Binary-Coded Hexadecimal
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001

Decimal	Hexadecimal	Binary-Coded Hexadecimal
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111
...	...	...
124	7C	0111 1100
...	...	...
365	16D	0001 0110 1101

The Hexadecimal Numbering System

Decimal	Hexadecimal		
	256	16	1
	16^2	16^1	16^0
0	0	0	0
1	0	0	1
2	0	0	2
3	0	0	3
4	0	0	4
5	0	0	5
6	0	0	6
7	0	0	7
8	0	0	8
9	0	0	9
10	0	0	A
11	0	0	B
12	0	0	C
13	0	0	D
14	0	0	E
15	0	0	F
16	0	1	0
17	0	1	1
...		...	
26	0	1	A

F	A	9	7
1 1 1 1	1 0 1 0	1 0 0 1	0 1 1 1

Data Organization, x86-32 (IA-32)

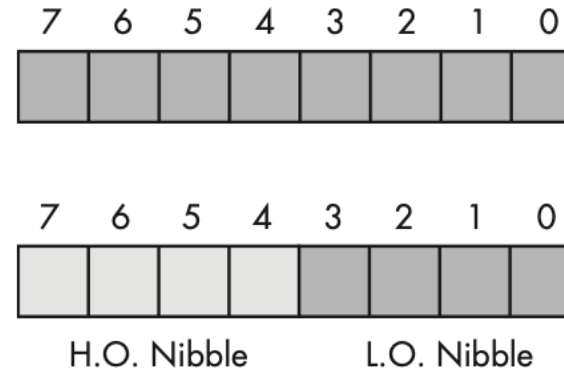
- In pure mathematics a value's representation may take require an arbitrary number of bits
- Computers generally work with some specific number of bits
- bits, nibbles, bytes, words, double words, quad words and long words

Data Organization, x86-32 (IA-32)

- Bits
 - The smallest unit of data on a binary computer is a single bit.
 - With 1 bit you can represent any two distinct items.
 - e.g.
 - To confuse things even more, different bits can represent different things.
 - How can you tell by looking at the bits?
 - Data is what you define it to be.
 - Single-bit values aren't the most popular data type you'll use
 - Most items you'll be trying to model require more than two different values

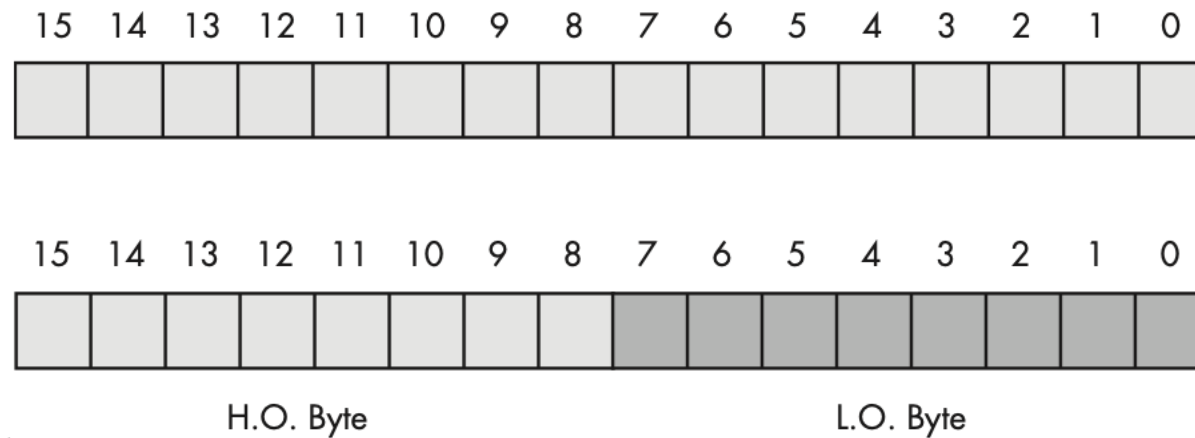
Data Organization, x86-32 (IA-32)

- Nibbles
 - A nibble is a collection of 4 bits.
 - hex numbers are represented on 4 bits
- Bytes
 - The most important data structure used by the x86-32 microprocessor is the byte
 - It consists of 8 bits



Data Organization, x86-32 (IA-32)

- Words
 - A word is a group of 16 bits.



Data representation

- Sign number
- representation **vs** interpretation!!!